

计算机体系结构

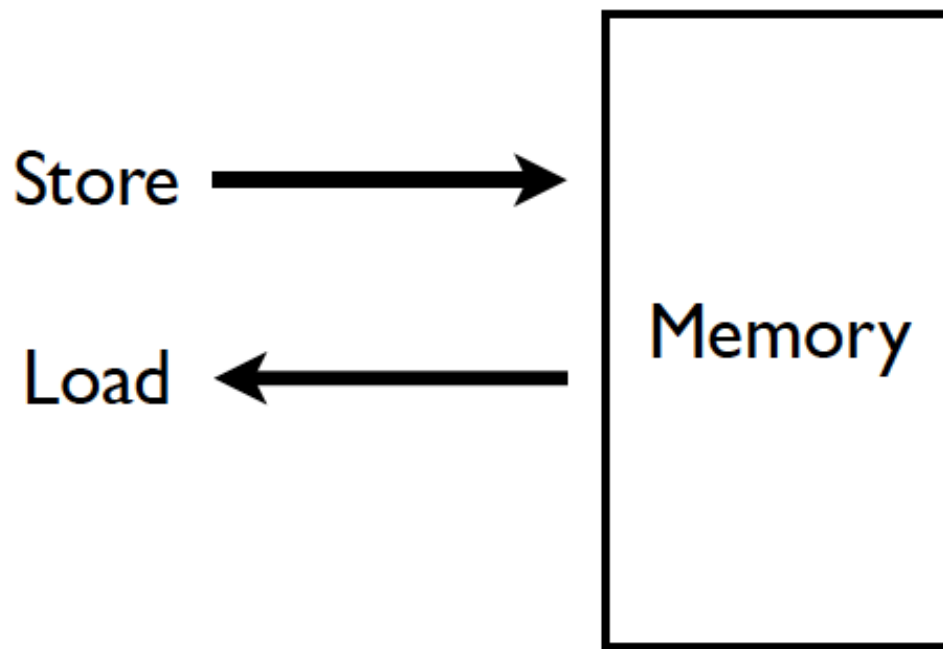
3. 存储层次架构

李建华

计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburgh, and eth.
The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

程序员视角下的存储器



- Q1：对Memory可能有哪些要求？

虚拟和物理存储器

- 当前的系统中，程序员看到的是虚拟的存储器。
 - 每个程序员认为存储器是**无限大**的，don't care ~
 - Q2：你编程的时候，考虑过内存或者存储的容量吗？
- 实际上，物理存储器的大小 远小于 程序员认为的大小。
- 系统 (软件+硬件) 动态地将 **虚拟地址** 映射到 **物理地址**，将物理存储器虚拟化。
 - 对程序员来说，存储器的管理是透明的。

虚拟存储器

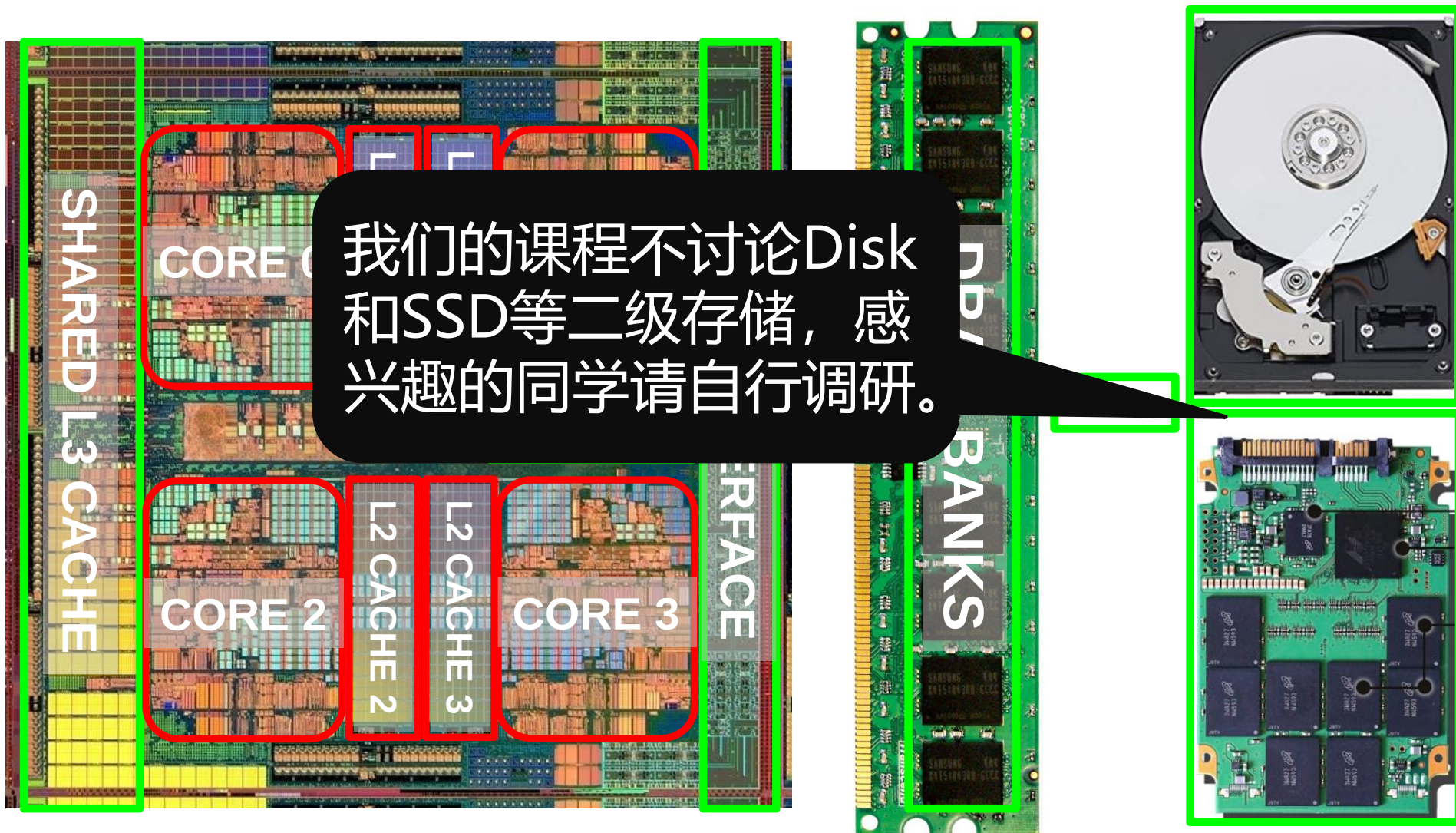
- **优点:**

- 对程序员来说, 不需要知道存储器的大小, 也不需要管理物理存储;
- 从系统层面来说, 一个较小的存储器可以作为一个巨大的存储器使用;
- 对程序员来说, life is easier, coding is fun and efficient.

- **缺点:**

- 更加复杂的软件和硬件: MMU和相应的中间件。

现代计算系统中的存储器

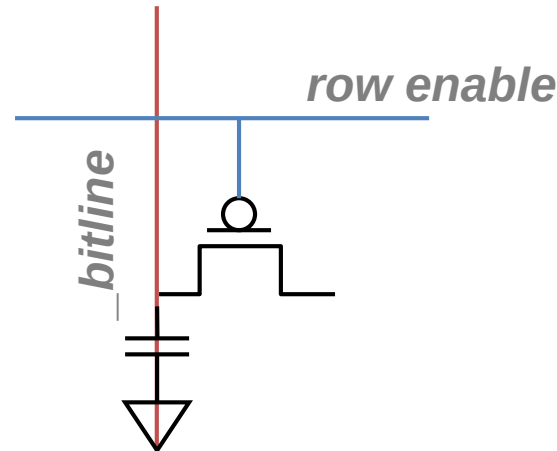
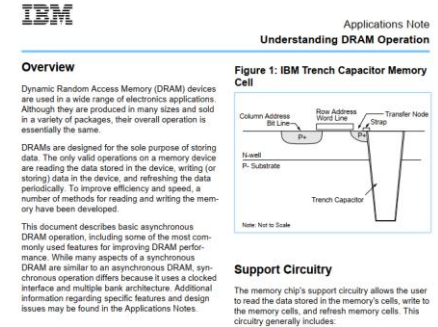


理想 vs 现实的冲突

- Bigger is slower
 - Bigger → 需要更长的时间来寻址（寻找目标）
- Faster is more expensive
 - 存储器技术: SRAM vs. DRAM vs. Disk vs. Tape
- Higher bandwidth is more expensive
 - 需要更多的Bank, 接口数, 工作频率, 或者说需要更快的技术
 - 通俗地说：配置越高，价格越贵。

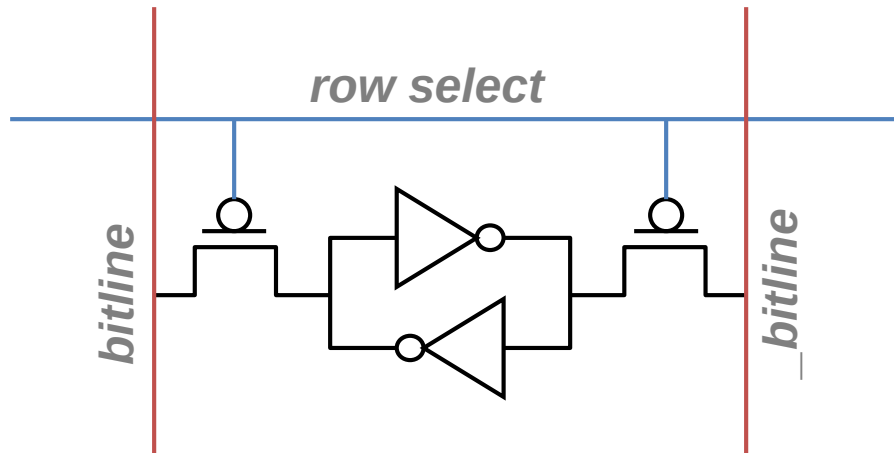
存储技术 - DRAM

- 电容里电荷的状态用来表明存储的信息
 - 电容的充电和放电状态表明1或0
 - 1 个电容
 - 1 个访问晶体管
- 电容会沿着RC路径泄露电荷
 - 或者说，DRAM存储单元随时间的推移会丢失电荷
 - 为了不丢失数据，DRAM的存储单元需要进行周期性（如64ms）的刷新



存储技术 - SRAM

- 用二个交叉耦合的反相器来存储一个bit
 - 反馈路径会使得保存的值一直稳定存在 “cell” 里
 - 4 个晶体管用于存储
 - 2 个晶体管用于访问



IBM

Applications Note Understanding Static RAM Operation

Overview

This document describes basic synchronous SRAM operation, including some of the most commonly used features for improving SRAM performance.

Fast, Faster, Fastest

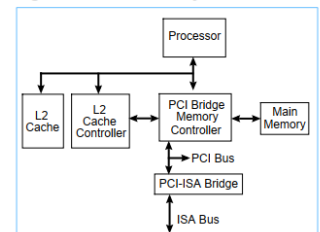
As microprocessors and other electronics applications get faster and faster, the need for large quantities of data at very high speeds increases, while providing the data at such high speeds gets more difficult to accomplish. As microprocessor speeds increase from 25 MHz to 100 MHz, to 250 MHz and beyond, systems designers have become more creative in their use of cache memory, interleaving, burst mode and other high-speed methods for accessing memory.

The old systems sporting just an on-chip instruction cache, a moderate amount of DRAM and a hard drive have given way to sophisticated designs using multilevel memory architectures. One of the primary building blocks of the multi-level memory architecture is the data cache.

cache is periodically off-loaded to the DRAM (extended or Level 3 memory) or to one of the disk drives.

Increasingly, the designers of high performance systems, including PC and RISC based computers and high speed telecommunications applications, rely on synchronous SRAMs to design caches that provide data at the speeds they require.

Figure 1. Basic Cache System



DRAM vs SRAM

- DRAM
 - 访问速度较慢 (capacitor)
 - 存储密度大 (1T-1C cell)
 - 低开销
 - 需要刷新 (功耗、性能、电路复杂度)
 - 生产需要将capacitor和CMOS逻辑电路放一起
- SRAM
 - 访问速度较快 (no capacitor)
 - 存储密度小 (6T cell)
 - 高开销
 - 不需要刷新
 - 生产过程与CMOS逻辑兼容 (no capacitor)

问题分析

- Bigger is slower
 - SRAM, 512 Bytes, sub-nanosec
 - SRAM, KByte~MByte, ~nanosec
 - DRAM, Gigabyte, ~50 nanosec
 - Hard Disk, Terabyte, ~10 millisec
- Faster is more expensive (dollars and chip area)
 - SRAM, < 10\$ per Megabyte
 - DRAM, < 1\$ per Megabyte
 - Hard Disk < 1\$ per Gigabyte
- 当前也有一些其它的新兴技术：
 - Flash memory, PC-RAM, STT-RAM, RRAM (not mature yet)

注意：这里的性能数据不是绝对的，会随着时间改变。

为什么采用存储层次架构？

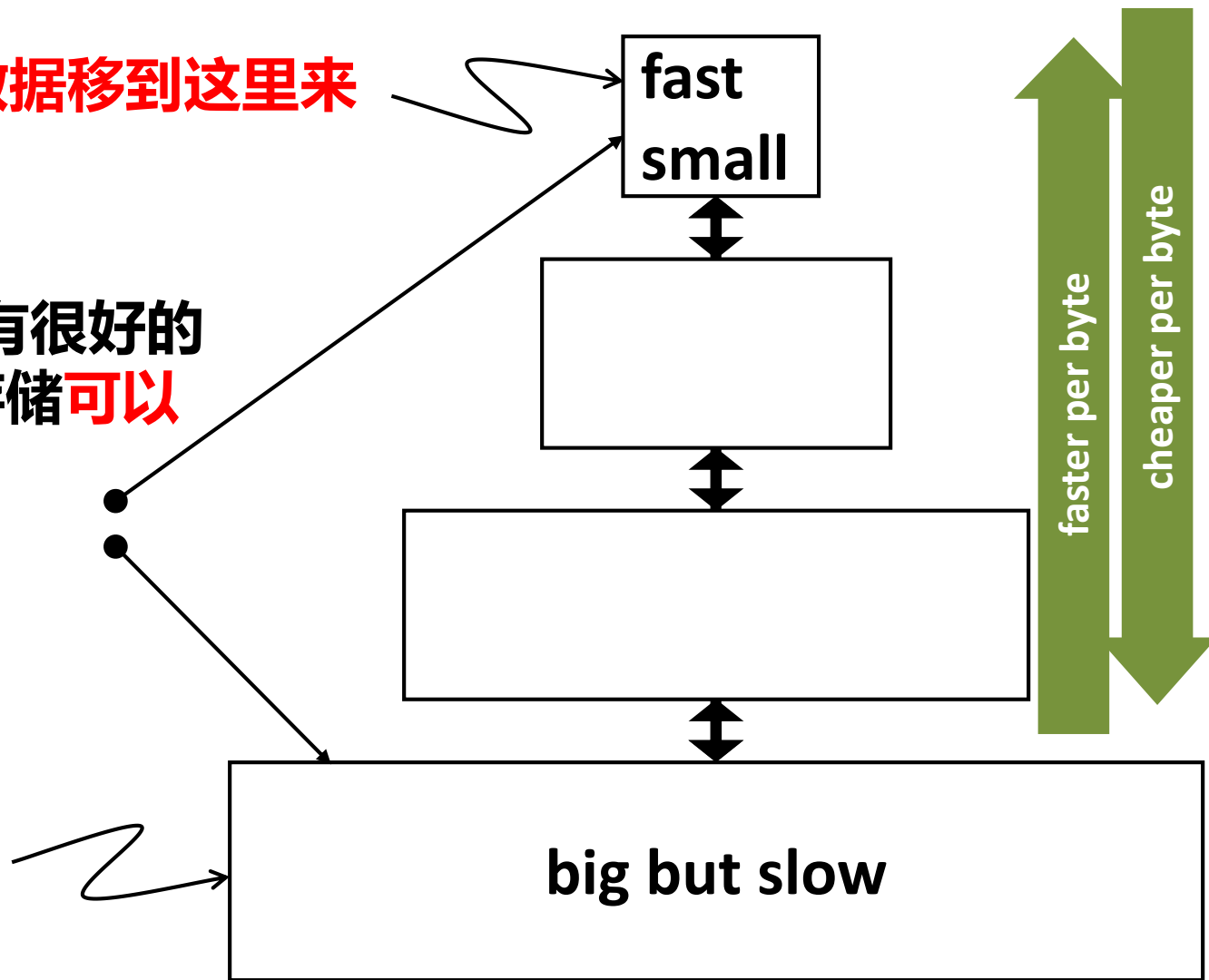
- 用户需要**速度快、容量大、价格便宜**的存储器！
 - 当然还有 高可靠性和 低功耗
- 但是，当前没有任何一种存储技术能同时满足上述三个目标。
- **解决方案：**使用多级存储器，采用不同的存储技术，也就是构建存储层次架构。
 - 存储级别离CPU越远，使用较大和较慢的存储技术；
 - 此外，确保CPU**当前**需要的大多数数据能够在较近且较快的存储级别里找到。

存储层次架构

将当前使用的数据移到这里来

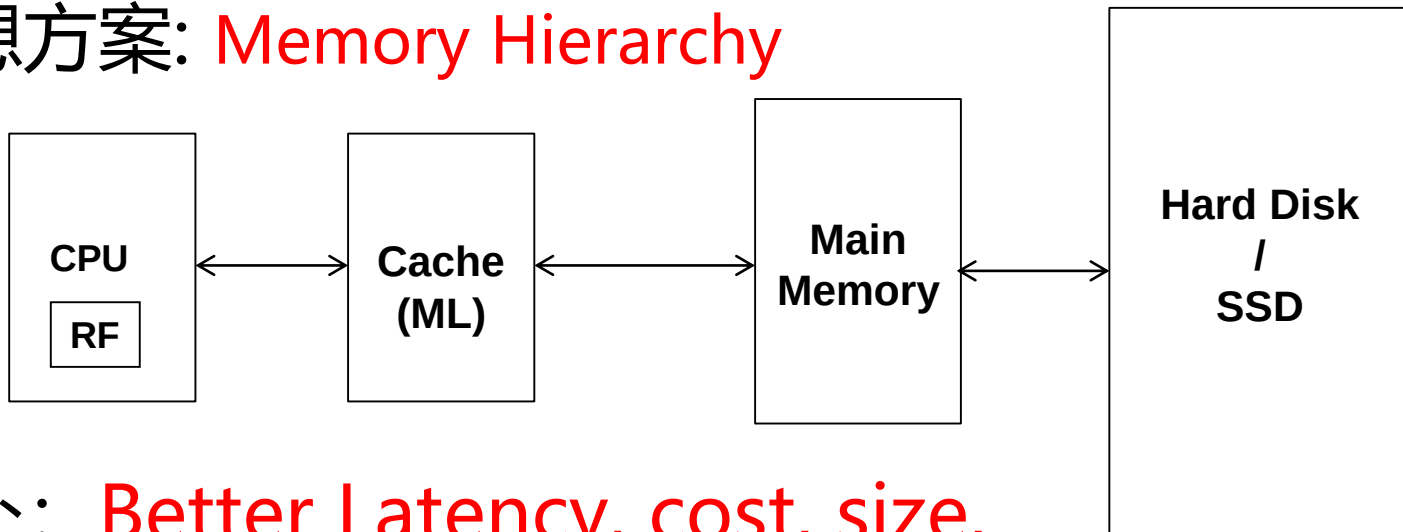
当访问行为具有很好的
局部性, 整个存储可以
既快又大。

将所有的指令和
数据在这里进行
备份



存储层次架构

- 基本的权衡
 - 快速存储: small, expensive
 - 大容量存储: slow, cheap
- 理想方案: Memory Hierarchy



- 好处: Better Latency, cost, size, bandwidth tradeoffs

Q3: Why MH works?

- **Temporal Locality:**
 - 当前所访问的指令或者数据不久后会被再次访问
- **Spatial Locality:**
 - 当前所访问的指令或者数据的周边数据，在不久的将来会被访问。

In computer science, **locality of reference**, also known as the **principle of locality**,^[1] is the tendency of a processor to access the same set of memory locations repetitively over a short period of time.^[2] There are two basic types of reference locality – temporal and spatial locality. Temporal locality refers to the reuse of specific data, and/or resources, within a relatively small time duration. Spatial locality refers to the use of data elements within relatively close storage locations. Sequential locality, a special case of spatial locality, occurs when data elements are arranged and accessed linearly, such as, traversing the elements in a one-dimensional array.

https://en.wikipedia.org/wiki/Locality_of_reference

Q4: Why locality exist?

- 典型程序的存储器访问行为存在大量局部性
 - 因为典型的程序由大量的 “**loops**” 和 耦合性高的 **数据结构** 构成
- Temporal: 一个程序倾向于**多次访问相同的存储位置**，并且是在一个小的时间窗口内。
 - 平均而言，一个程序80%以上的时间在处理**循环**。
- Spatial: 一个程序倾向于**一次连续访问一组相邻的存储位置**。
 - 例如指令存储器的访问，通常情况下是顺序访问；
 - 数组或数据结构的访问，具有顺序访问的特征。

高速缓存溯源 (1)

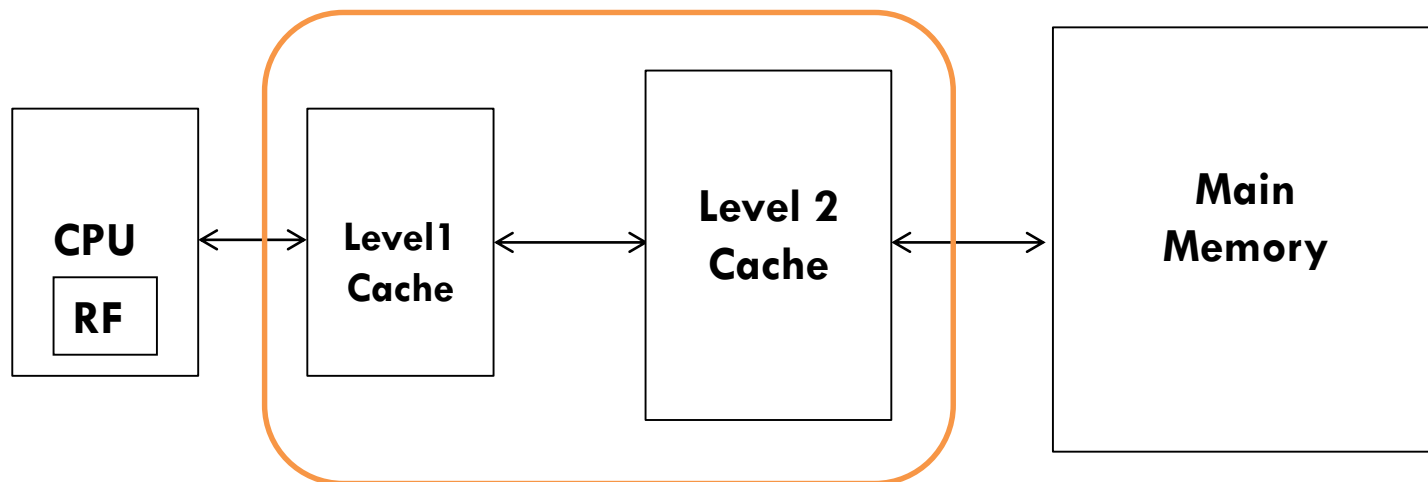
- **想法1:** 将最近访问的数据存放于一个**自动管理的快速存储里** (called cache)
- **期望:** 所保存的数据在不久会被再次访问
- Temporal locality 原理
 - 首次在[Maurice Wilkes](#)的论文里提出:
 - Wilkes, “**Slave Memories and Dynamic Storage Allocation**,” IEEE Trans. on Electronic Computers, 1965.
 - “The use is discussed of a fast core memory of, say 32000 words as a slave to a slower core memory of, say, one million words **in such a way that in practical cases the effective access time is nearer that of the fast memory than that of the slow memory.**”

高速缓存溯源 (2)

- **想法2:** 将地址相邻于当前所访问的数据的内容保存在一个自动管理的快速存储器中
 - 将存储器逻辑地划分为大小相同的块
 - 将访问的块整个取到高速缓存里
- **期望:** 附近的数据在不久会被访问
- Spatial locality原理
 - 首次在IBM 360/85里实现 [Product]
 - 16 Kbyte cache with 64 byte blocks
 - Liptay, “Structural aspects of the System/360 Model 85 II: the cache,” IBM Systems Journal, 1968.

Why multi-level cache?

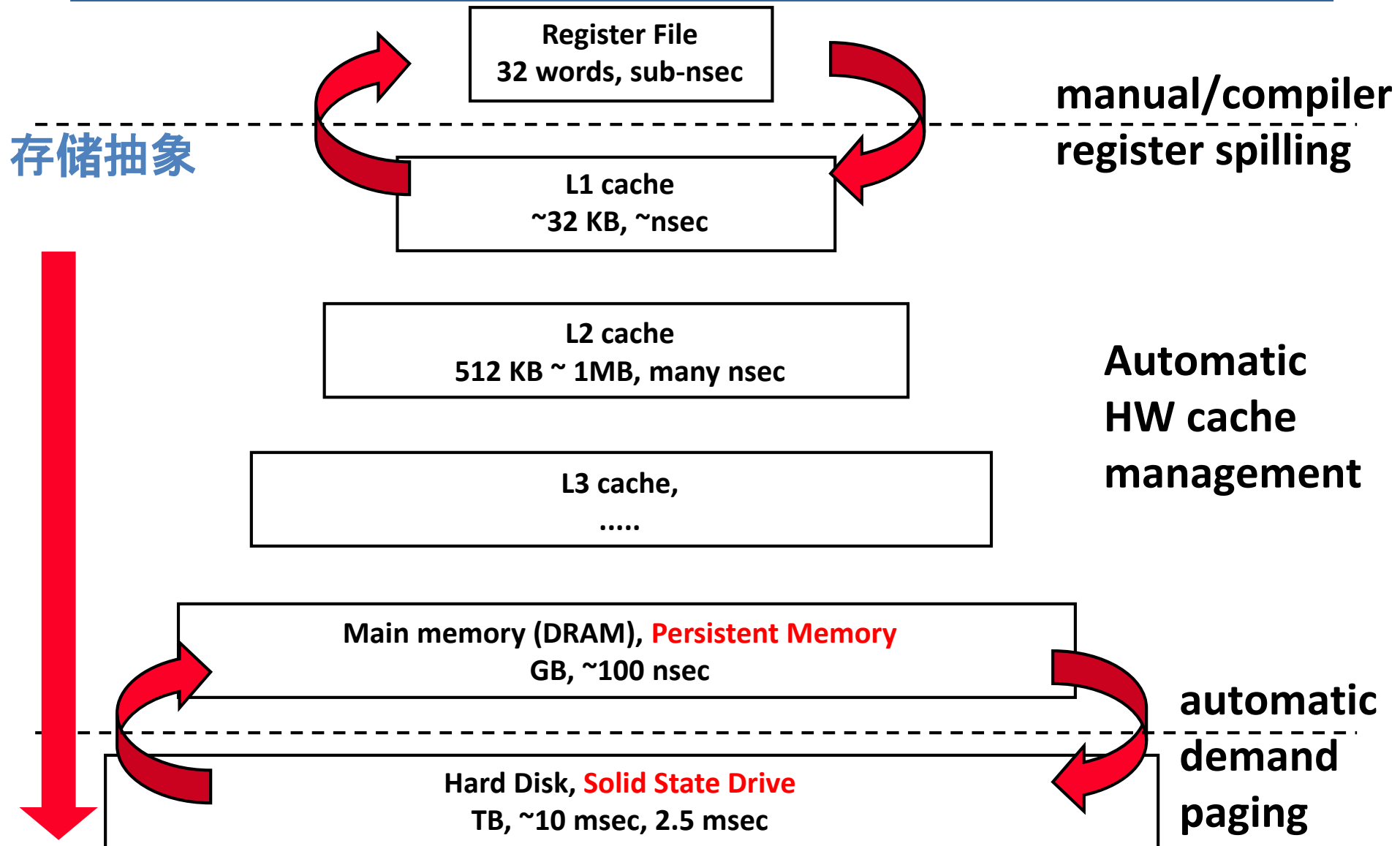
- 高速缓存需要非常紧凑地整合到流水线中
 - 理想情况下, 访问缓存只能有一个周期的访问延迟, 相关的操作不会影响/暂停流水线的处理。
- 若流水线工作频率很高 → 配置的高速缓存不能大
 - 但是, 我们想同时拥有 大容量缓存 和 快速流水线处理!
 - 那么, 该如何解决这个问题呢?
- 方案: 缓存层次架构



高速缓存的管理

- **Manual:** 程序员管理不同存储层级中的数据移动
 - 对大体量的程序来说，程序员很痛苦；
 - 上世纪50年代的 “core” vs “drum” 存储
 - 一些嵌入式系统中的存储依然如此 (scratch pad)
- **Automatic:** 硬件管理不同存储层级间的数据移动，对程序员来说是透明的。
 - ++ 程序员可以轻松一些
 - ++ 很多程序员甚至都不需要知道这个事情的存在
 - 对于写一个**正确的**程序来说，你不需要知道缓存多大以及如何工作！
 - 但是，如果你想写出一个运行速度很快的程序，则需要掌握缓存。
。 [Key programmer requirement]

现代的存储层次架构



访存性能 (延迟) 分析

- 对于一个给定的存储层级: i
 - 命中时, 经历一个由存储技术和配置决定的访问延迟: t_i
 - 缺失时, 感知到访问该存储层级的延迟为: T_i (T_i 大于 t_i)
- 除了最外层的存储层级, 当你根据一个地址在某个存储层级去查找目标的时候:
 - 存在一个命中的概率: h_i , 相应的访问时间为: t_i
 - 存在一个缺失的概率: m_i , 相应的访问时间为: $t_i + T_{i+1}$
 - 其中: $h_i + m_i = 1$
- 因此:

$$T_i = h_i \cdot t_i + m_i \cdot (t_i + T_{i+1})$$

$$T_i = t_i + m_i \cdot T_{i+1}$$

访存性能分析（绪）

- 递推的访存延迟计算方法：

$$T_i = t_i + m_i \cdot T_{i+1}$$

- 目标：在可允许的开销下，达到预期的 T_1 。
- 期望： $T_i \approx t_i$ ，那么可能得优化方向有2个：
 - 方向一：Keep m_i low
 - 提高容量 C_i 会降低 m_i ，但是有可能增加 t_i
 - 通过智能的管理方法来降低 m_i (如，替换策略: 猜测你不要的, 预取: 猜测你需要的)
 - 方向二：Keep T_{i+1} low
 - 更快的高层级存储设计，但是要小心增加开销
 - 通过引入中间层级来做一个折中（多级缓存）

高速缓存基础

B.1	Introduction	B-2
B.2	Cache Performance	B-16
B.3	Six Basic Cache Optimizations	B-22
B.4	Virtual Memory	B-40
B.5	Protection and Examples of Virtual Memory	B-49
B.6	Fallacies and Pitfalls	B-57
B.7	Concluding Remarks	B-59
B.8	Historical Perspective and References	B-59
	Exercises by Amr Zaky	B-60

- **Read the Appendix-B carefully by yourself!**
- **Because we cannot cover all details in class.**

“高速” 缓存简介 (1)

- 通俗地说, 任何能 **记住/保存** 频繁使用的数据 的结构来避免长延时的重新获取这些数据的操作, 就叫缓存。
 - 例如: Web缓存, 手机APP里的缓存等。
- 我们讨论的对象是片上缓存: 一个基于**SRAM**的由硬件自动管理的存储层级。
 - 将 DRAM/下一级缓存 中频繁访问的内容保存在 SRAM里, 来避免重复访问DRAM的长延时开销。

高速缓存简介 (2)

- Cache block (line): 缓存中的存储粒度
 - Memory is logically divided into cache blocks that map to locations in the cache
- 涉及高速缓存的一些关键问题：
 - **查找与放置**: 如何查找和放置一个Block?
 - **替换**: 缓存满了之后, 如何选择一个Victim?
 - **缓存空间的划分粒度**: Block的大小如何设置?
 - **写策略**: 对于写操作, 如何处理?

请认真阅读下面的内容

2.1	Introduction	72
2.2	Ten Advanced Optimizations of Cache Performance	78
2.3	Memory Technology and Optimizations	96
2.4	Protection: Virtual Memory and Virtual Machines	105
2.5	Crosscutting Issues: The Design of Memory Hierarchies	112
2.6	Putting It All Together: Memory Hierarchies in the ARM Cortex-A8 and Intel Core i7	113
2.7	Fallacies and Pitfalls	125
2.8	Concluding Remarks: Looking Ahead	129
2.9	Historical Perspective and References	131
	Case Studies and Exercises by Norman P. Jouppi, Naveen Muralimanohar, and Sheng Li	131